

Nome: Guilherme de Moraes China Costa

Data: 20/05/2026

Título: Design Patterns e Domain-Driven Design

repo:

https://github.com/guilhermecosta/INFNET/tree/master/design_patterns_ddd/TP2

* No repo também tem um readme com as explicações abaixo.

1. Gestão da Complexidade

O Domain-Driven Design ajuda a operadora logística a crescer sem transformar o sistema em um bloco monolítico confuso, porque separa o negócio em contextos claros, com linguagens e responsabilidades bem definidas. Em vez de misturar regras de frete, rastreamento, faturamento e manutenção no mesmo fluxo técnico, o DDD permite modelar cada área conforme sua especialidade, reduzir acoplamento, alinhar desenvolvedores com especialistas de negócio e facilitar a evolução do ecossistema quando surgirem novos canais de entrega, regras fiscais, integrações bancárias ou tipos de veículo.

2. Domínios e Subdomínios

Domínio principal

- Gestão de Fretes (Core Domain): concentra a geração de valor principal da operadora, pois calcula rotas, seleciona modal de envio e define o preço do frete.

Subdomínios de suporte

- Rastreamento (Supporting Subdomain): apoia a operação com visibilidade em tempo real da entrega.
- Faturamento (Supporting Subdomain): sustenta a monetização e a conformidade fiscal da operação.
- Manutenção de Frota (Supporting Subdomain): garante disponibilidade operacional dos veículos e reduz paradas não planejadas.

Subdomínios genéricos

- Consolidação de dados cadastrais (Generic Subdomain): identificadores, endereços, placas, documentos e contatos.
- Auditoria e notificações (Generic Subdomain): trilhas de evento, alertas e comunicação com usuários e operadores.

3. Bounded Contexts

Definição dos contextos

Bounded Context	Responsabilidades
Fretes	Cotação, roteirização, seleção de modal, cálculo de prazo e preço
Rastreamento	Eventos de localização, status da entrega, checkpoint e previsão de chegada
Faturamento	Emissão de nota fiscal, cobrança, conciliação bancária e fechamento financeiro
Manutenção de Frota	Plano preventivo, ordem de serviço, histórico de manutenção e indisponibilidade de veículos
Shared Kernel	Tipos compartilhados entre contextos: <code>EntregaId</code> , <code>FreteId</code> , <code>VeiculoId</code> , <code>Endereco</code> , <code>ModalTransporte</code> , <code>StatusEntrega</code> , <code>Dinheiro</code>

Relação entre contextos

- [Frete] — Published Language —> [Rastreamento]
- [Frete] — Customer/Supplier —> [Faturamento]
- [Manutenção] — Published Language —> [Frete]
- [Rastreamento] — Customer/Supplier —> [Faturamento]
- Todos os contextos consomem tipos do [Shared Kernel] via dependência direta.

4. Linguagem Ubíqua

Glossário obrigatório para desenvolvedores e especialistas de negócio:

#	Termo	Definição
1	Frete	Serviço de transporte contratado para mover uma carga entre origem e destino.
2	Entrega	Execução operacional do frete, acompanhada até a confirmação final pelo destinatário.
3	Modal de Transporte	Tipo de envio utilizado: motoboy, navio, trem ou caminhão.
4	Rota	Percurso planejado para cumprir a entrega, podendo envolver múltiplos trechos e modais.
5	Cotação de Frete	Simulação de preço e prazo gerada antes da contratação efetiva do serviço.
6	Checkpoint	Registro intermediário de localização ou mudança de status ao longo da entrega.
7	Status da Entrega	Situação atual da entrega: PLANEJADA, EM_TRANSITO, ATRASADA ou CONCLUIDA.

8	Nota Fiscal	Documento fiscal emitido a partir do serviço de frete prestado.
9	Conciliação Bancária	Validação entre cobranças emitidas e recebimentos efetivamente confirmados.
10	Ordem de Manutenção	Registro de intervenção preventiva ou corretiva em um veículo da frota.
11	Indisponibilidade de Veículo	Período em que um veículo não pode ser escalado para novas entregas.
12	Previsão de Entrega	Data e hora estimadas para conclusão da entrega, recalculada a cada checkpoint.

5. Design Estratégico e Padrões de Integração

Shared Kernel

O Shared Kernel deve ser pequeno e muito estável, contendo apenas conceitos realmente compartilhados entre os contextos. Neste cenário, ele reúne:

- Identificadores de negócio: `Entregald`, `Freteld`, `Veiculold`
- Objetos de valor comuns: `Endereco`, `Localizacao`, `Periodo`, `Dinheiro`
- Enumerações padronizadas: `ModalTransporte`, `StatusEntrega`

Esses elementos vivem no shared kernel porque aparecem em mais de um contexto e precisam manter significado idêntico em toda a solução. O cuidado estratégico é nunca compartilhar regras de negócio específicas — o kernel deve carregar apenas tipos estáveis.

Padrões de integração entre contextos

Os padrões abaixo fazem parte do Design Estratégico do DDD e definem como os contextos se relacionam, não apenas que se relacionam:

1. Published Language — Manutenção → Fretes

Padrão: Published Language (Linguagem Publicada)

A Manutenção publica eventos de indisponibilidade de veículos por meio de uma linguagem formal e documentada (ex.: eventos de domínio ou contratos de API versionados). O contexto de Fretes consome essa linguagem sem depender dos internos de Manutenção.

Por quê? Os dois contextos têm modelos de Veiculo com focos diferentes — Manutenção cuida do histórico técnico, Fretes cuida da capacidade operacional. A Published Language evita que um contexto invada o modelo do outro.

2. Published Language — Fretes → Rastreamento

Padrão: Published Language (Linguagem Publicada)

Frete publica o evento EntregaIniciada (contendo EntregaId, Rota e ModalTransporte) via mensageria ou webhook. Rastreamento consome esse contrato sem precisar conhecer as regras de cotação.

Por quê? Rastreamento não precisa saber como o frete foi calculado — apenas que uma entrega existe e precisa ser monitorada. A separação evita acoplamento entre o core domain e o subdomain de suporte.

3. Customer/Supplier — Fretes → Faturamento

Padrão: Customer/Supplier (Cliente/Fornecedor)

Frete é o Supplier (fornecedor) e Faturamento é o Customer (cliente). Frete se compromete a manter a interface de publicação do serviço prestado e notifica Faturamento quando uma entrega é concluída. Faturamento pode influenciar a evolução dessa interface, mas não a controla.

Por quê? Faturamento depende criticamente de dados de Frete (valor, origem, destino, modalidade) para emitir a nota fiscal. A relação é assimétrica e direcionada — por isso Customer/Supplier, não Partnership.

4. Customer/Supplier — Rastreamento → Faturamento

Padrão: Customer/Supplier (Cliente/Fornecedor)

Rastreamento confirma o StatusEntrega = CONCLUIDA, o que é o gatilho para Faturamento iniciar o processo de emissão da nota fiscal. Faturamento é o Customer que aguarda esse sinal.

Por quê? O faturamento só deve ocorrer após a confirmação operacional da entrega. Rastreamento é a fonte autoritativa desse status — Faturamento não deve inferir conclusão por outros meios.

Resumo dos padrões

Relação	Padrão	Direção
Manutenção → Fretes	Published Language	Manutenção publica, Fretes consome
Frete → Rastreamento	Published Language	Frete publica, Rastreamento consome
Frete → Faturamento	Customer/Supplier	Frete (Supplier) serve Faturamento (Customer)
Rastreamento → Faturamento	Customer/Supplier	Rastreamento (Supplier) serve Faturamento (Customer)